

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1017	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Mukesh Raj L	Reg. No.:	192571036
Date:		Duration:	60 minutes

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Analysis.

1.1 Problem Statement

Pharmaceutical manufacturing requires strict quality assurance to ensure that medicines are safe, effective, and produced according to required quality and regulatory standards. Traditional quality assurance processes depend heavily on manual inspection, periodic monitoring, and documentation. These methods are time-consuming, labor-intensive, and may result in inconsistencies and delayed identification of manufacturing defects.

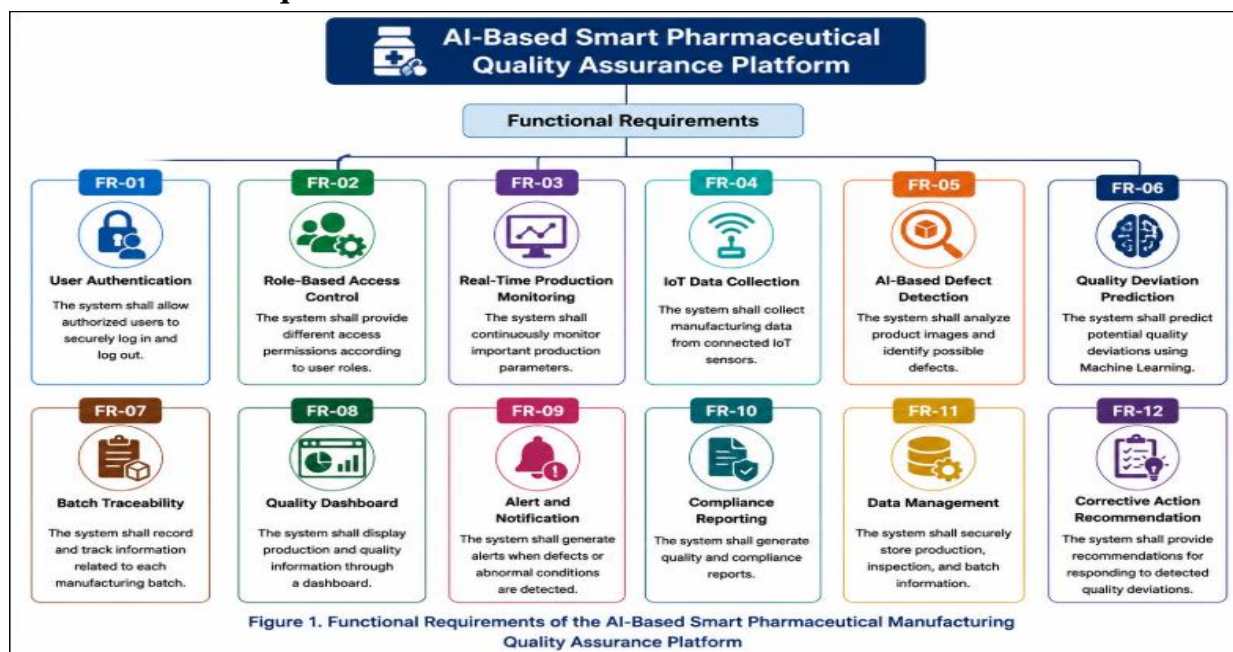
Pharmaceutical companies may also face production delays, quality inconsistencies, increased product wastage, and higher operational costs when defects are detected only after the production process. Manual documentation and fragmented quality management systems can make it difficult to maintain complete batch information and prepare quality and compliance reports efficiently.

The proposed **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform** addresses these challenges by integrating **Artificial Intelligence (AI), Computer Vision, Internet of Things (IoT), Machine Learning, and Cloud Computing**. The system continuously monitors manufacturing parameters, detects product defects using AI-based image analysis, predicts possible quality deviations, maintains batch traceability, generates quality and compliance reports, and provides alerts when abnormal conditions are detected.

1.2 Project Objectives

1. To continuously monitor pharmaceutical manufacturing processes.
2. To collect production parameters using IoT sensors.
3. To detect product defects automatically using Artificial Intelligence and Computer Vision.
4. To predict potential quality deviations using Machine Learning.
5. To maintain complete traceability of pharmaceutical production batches.
6. To generate quality and compliance reports efficiently.
7. To provide real-time alerts when abnormal production conditions are detected.
8. To recommend suitable corrective actions for identified quality problems.
9. To reduce product wastage and manufacturing losses.
10. To improve pharmaceutical manufacturing quality, productivity, and operational efficiency.

1.3 Functional Requirements



The **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform** shall provide the following functional requirements:

1. **User Authentication** – The system shall allow authorized users to securely log in and log out.
2. **Role-Based Access Control** – The system shall provide different access permissions based on user roles such as administrator, quality manager, production manager, and operator.
3. **Real-Time Production Monitoring** – The system shall continuously monitor important pharmaceutical manufacturing parameters.
4. **IoT Data Collection** – The system shall collect production data from connected IoT sensors installed in the manufacturing environment.
5. **AI-Based Defect Detection** – The system shall analyze product images using Computer Vision and identify defects in pharmaceutical products and packaging.
6. **Quality Deviation Prediction** – The system shall use Machine Learning to predict potential quality deviations based on production and historical data.
7. **Batch Traceability** – The system shall record and track information associated with each pharmaceutical production batch.
8. **Quality Dashboard** – The system shall display production status, quality results, defect information, and important performance indicators through a centralized dashboard.
9. **Alert and Notification System** – The system shall generate alerts when abnormal production conditions or product defects are detected.
10. **Compliance Reporting** – The system shall generate quality and compliance reports from stored production and inspection information.
11. **Data Management** – The system shall store production data, inspection results, batch records, and quality reports for future reference.
12. **Corrective Action Recommendation** – The system shall provide recommendations to quality personnel when quality deviations or defects are identified.

1.4 Expected Outcomes

The proposed **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform** is expected to produce the following outcomes:

1. **Improved Product Quality** – AI-based inspection and continuous monitoring help identify quality issues and maintain consistent pharmaceutical product quality.
2. **Reduced Manufacturing Defects** – Computer Vision can identify visible product defects during the manufacturing process, reducing the chances of defective products reaching later stages.
3. **Faster Quality Inspections** – Automated inspection reduces dependence on manual checking and helps quality teams identify problems more quickly.
4. **Lower Production Costs** – Early detection of defects can reduce material wastage, rework, and unnecessary production losses.
5. **Better Regulatory Compliance** – Automated quality records and compliance reports support organized documentation and regulatory requirements.
6. **Enhanced Operational Efficiency** – Real-time monitoring and centralized quality information help production and quality teams make faster decisions.
7. **Increased Patient Safety** – Improved quality monitoring helps reduce the possibility of defective or non-conforming pharmaceutical products reaching patients.
8. **Improved Manufacturing Productivity** – Automation of inspection, monitoring, reporting, and quality analysis can improve overall manufacturing productivity.

1.5 Proposed Solution

The proposed **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform** is designed to automate and improve the quality assurance process in pharmaceutical manufacturing. The system combines **IoT, Artificial Intelligence, Computer Vision, Machine Learning, and cloud-based data management** to provide continuous monitoring and intelligent quality analysis.

IoT sensors collect important production parameters from manufacturing equipment and the production environment. At the same time, AI-enabled cameras capture images of pharmaceutical products and packaging for automated visual inspection.

The collected data is processed by the backend and AI services. The Computer Vision module identifies visible product defects, while the Machine Learning module analyzes production data to predict possible quality deviations. When an abnormal condition or defect is detected, the system generates an alert for the responsible quality or production personnel.

All production, inspection, and batch information is stored in a centralized database. A web-based dashboard allows authorized users to view real-time production conditions, quality results, defect information, alerts, and reports. The platform also supports batch traceability and automated quality and compliance documentation.

Overall, the proposed solution helps identify quality problems earlier, reduce product wastage, improve manufacturing efficiency, and support better quality management.

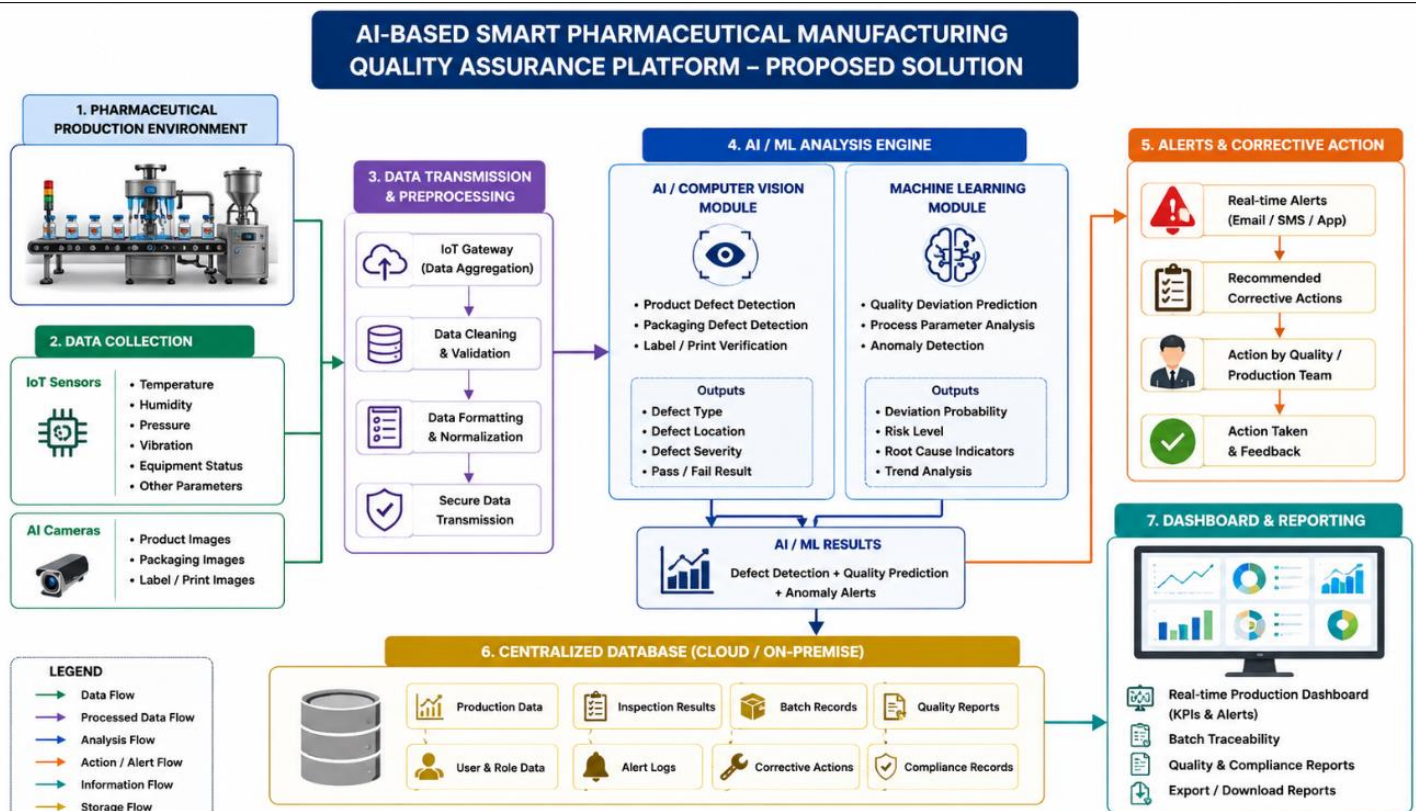


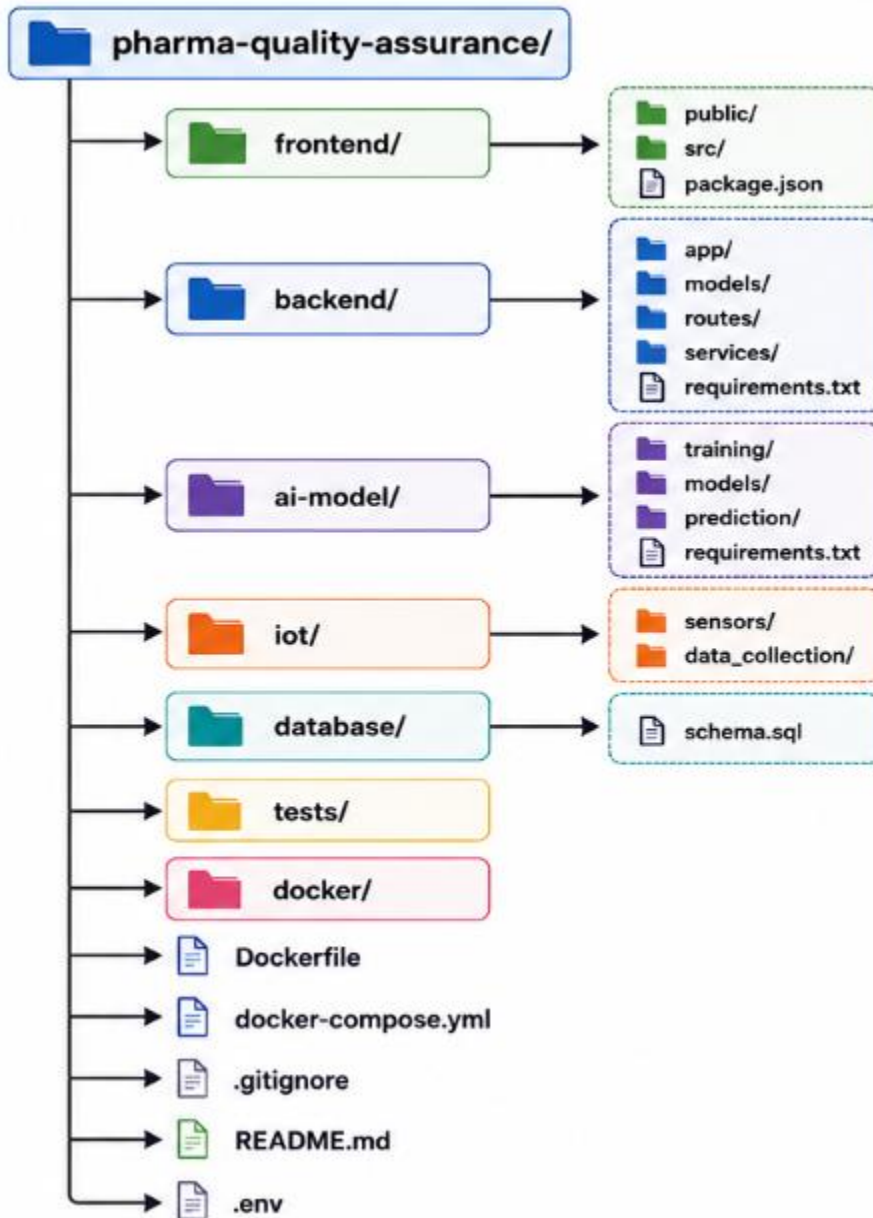
Figure 2: Proposed Solution Workflow of the AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform

2. Project Structure Design

The project repository is organized into separate folders and files so that the **frontend, backend, AI/ML module, IoT module, database, testing, and Docker configuration** can be developed and maintained systematically.

This section addresses **Question 2: Design the project structure**, as required in your uploaded Git & Docker assignment.

2.1 Proposed Repository Structure



2.2 Purpose of Major Folders and Files

Folder/File	Purpose
frontend/	Contains the web interface and quality monitoring dashboard.
backend/	Contains APIs, business logic, authentication, and application services.
ai-model/	Contains AI/ML training, trained models, and prediction modules.
iot/	Contains IoT sensor integration and production data collection components.
database/	Contains database schema and database-related scripts.
tests/	Contains test cases for validating different system components.
docker/	Contains additional Docker-related configuration files.
Dockerfile	Contains instructions for building the application container image.
docker-compose.yml	Defines and manages multiple application containers.
.gitignore	Prevents unnecessary and sensitive files from being tracked by Git.
README.md	Provides project description, setup instructions, and usage information.
.env	Stores environment-specific configuration values such as database settings.

2.3 Repository Structure Justification

The proposed structure separates the major components of the pharmaceutical quality assurance platform into independent modules. This makes the project easier to understand, develop, test, and maintain.

For example, the **AI model** can be updated without modifying the frontend, while the **IoT module** can independently handle sensor data collection. The backend provides a central connection between these modules and the user dashboard.

The structure also supports Git-based collaborative development because different developers can work on different modules using separate branches. In addition, the separation of application components makes it easier to containerize the system using Docker.

Folder / File	Purpose
 frontend/	Contains the web interface and quality monitoring dashboard.
 backend/	Contains APIs, business logic, authentication, and application services.
 ai-model/	Contains AI/ML training, trained models, and prediction modules.
 iot/	Contains IoT sensor integration and production data collection components.
 database/	Contains database schema and database-related scripts.
 tests/	Contains test cases for validating different system components.
 docker/	Contains additional Docker-related configuration files.
 Dockerfile	Contains instructions for building the application container image.
 docker-compose.yml	Defines and manages multiple application containers.
 .gitignore	Prevents unnecessary and sensitive files from being tracked by Git.
 README.md	Provides project description, setup instructions, and usage information.
 .env	Stores environment-specific configuration values such as database settings.

2.4 Repository Organization

The repository is organized to keep each major part of the **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform** separate and manageable. This organization makes development easier and supports Git-based collaboration and Docker-based deployment.

The **frontend** contains the user interface and quality monitoring dashboard. The **backend** contains APIs, authentication, business logic, and application services. The **AI-model** directory contains the Machine Learning training and prediction components, while the **IoT** directory manages sensor data collection. The **database** directory contains database-related scripts, and the **tests** directory is used to validate system functionality.

The root directory contains important project configuration files such as the Dockerfile, docker-compose.yml, .gitignore, and README.md. This structure allows developers to work on individual modules independently while maintaining a well-organized and maintainable repository.

Advantages of the Repository Organization

- Separates different system components clearly.
- Makes the project easier to understand and maintain.
- Supports independent development of modules.
- Simplifies Git branching and collaboration.
- Makes Docker containerization easier.
- Helps organize testing and deployment files.
- Supports future expansion of the system.

3. Git Repository Initialization

Git is used as the version control system for the project. It helps track changes, maintain the project history, create branches for different features, and support collaboration among developers.

For the **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform**, Git provides a structured way to manage the frontend, backend, AI, IoT, database, testing, and Docker-related files.

3.1 Repository Initialization

First, navigate to the project folder in the terminal and initialize Git using:

```
git init
```

This command creates a hidden .git directory inside the project folder. The .git directory stores the information required by Git to track changes and maintain the repository history.

3.2 Configure Git User Information

The developer's name and email address can be configured using:

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```

This information is associated with the commits created by the developer.

3.3 Check Repository Status

The current status of the project can be checked using:

```
git status
```

This command displays modified files, untracked files, and staged changes.

3.4 Add Project Files

All project files can be added to the staging area using:

```
git add .
```

The staging area allows the developer to select the changes that will be included in the next commit.

3.5 Create Initial Commit

The first version of the project can be saved using:

```
git commit -m "Initial project setup"
```

This creates the first entry in the Git project history.

3.6 Connect to Remote Repository

If a GitHub repository has been created, it can be connected using:

```
git remote add origin <repository-url>
```

The remote repository can then be checked using:

```
git remote -v
```

3.7 Push the Project

The project can be uploaded to the remote repository using:

```
git branch -M main  
git push -u origin main
```

This transfers the committed project files to the remote main branch.

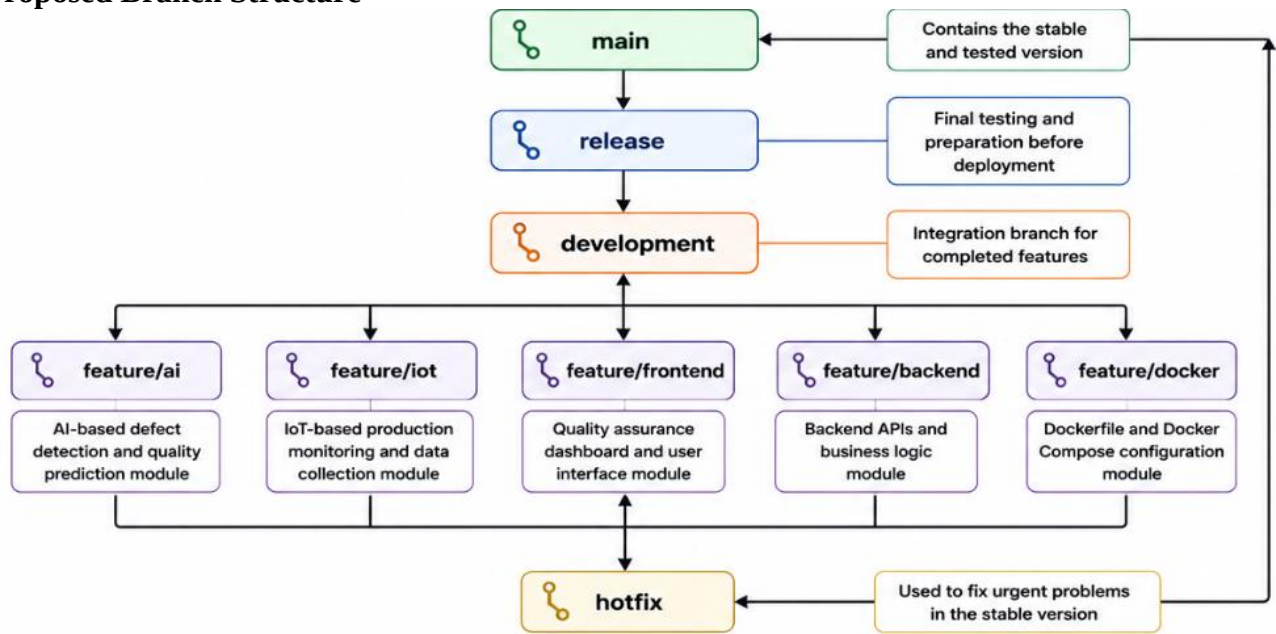
3.8 Importance of Git Repository Initialization

Initializing Git at the beginning of the project provides a foundation for version control. It allows the development team to track changes, maintain different versions of the project, create feature branches, recover previous versions, and collaborate through a remote repository.

4. Git Branching Strategy

A suitable branching strategy is required to manage the development of different modules of the **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform**. The proposed strategy separates stable code from ongoing development and feature-specific work.

4.1 Proposed Branch Structure



4.2 Branches and Their Purpose

Branch	Purpose
main	Contains the stable and tested version of the project.
development	Used to integrate completed features before the final release.
feature/ai	Used for developing AI-based defect detection and quality prediction.
feature/iot	Used for developing IoT-based production monitoring.
feature/frontend	Used for developing the quality assurance dashboard and user interface.
feature/docker	Used for Dockerfile and Docker Compose development.
release	Used for final testing and preparation before deployment.
hotfix	Used to fix urgent problems found in the stable version.

4.3 Feature Branch Development

Each major feature is developed in its own branch. For example, the AI module can be developed using:

```
git checkout developmentgit checkout -b feature/ai
```

After completing the AI functionality, the changes can be committed using:

```
git add .git commit -m "Add AI defect detection module"
```

The feature branch can then be pushed to the remote repository:

```
git push origin feature/ai
```

After testing and review, the feature can be merged into the development branch.

```
git checkout development
git merge feature/ai
```

4.4 Justification for the Branching Strategy

This branching strategy is suitable for the project because the platform contains several independent components such as AI, IoT, frontend, backend, and Docker configuration. Developers can work on these components separately without directly affecting the stable main branch.

The development branch provides an integration area where completed features can be tested together. The release branch can be used for final testing, while the hotfix branch allows urgent corrections to be handled separately.

This approach improves **collaboration, change management, code stability, testing, and release control.**

5. Version Control Workflow

The Git version control workflow defines how changes are created, tracked, committed, shared, reviewed, and integrated into the **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform**. A structured workflow helps maintain code quality and prevents unstable changes from affecting the main project.

5.1 Git Workflow Process

The proposed workflow follows these stages:

Create Branch → Develop Feature → Check Changes → Stage Changes → Commit → Push → Review → Merge → Test

Each feature is first developed in an appropriate feature branch. After development, the changes are reviewed and committed with meaningful messages. The completed feature is then merged into the `development` branch for integration testing.

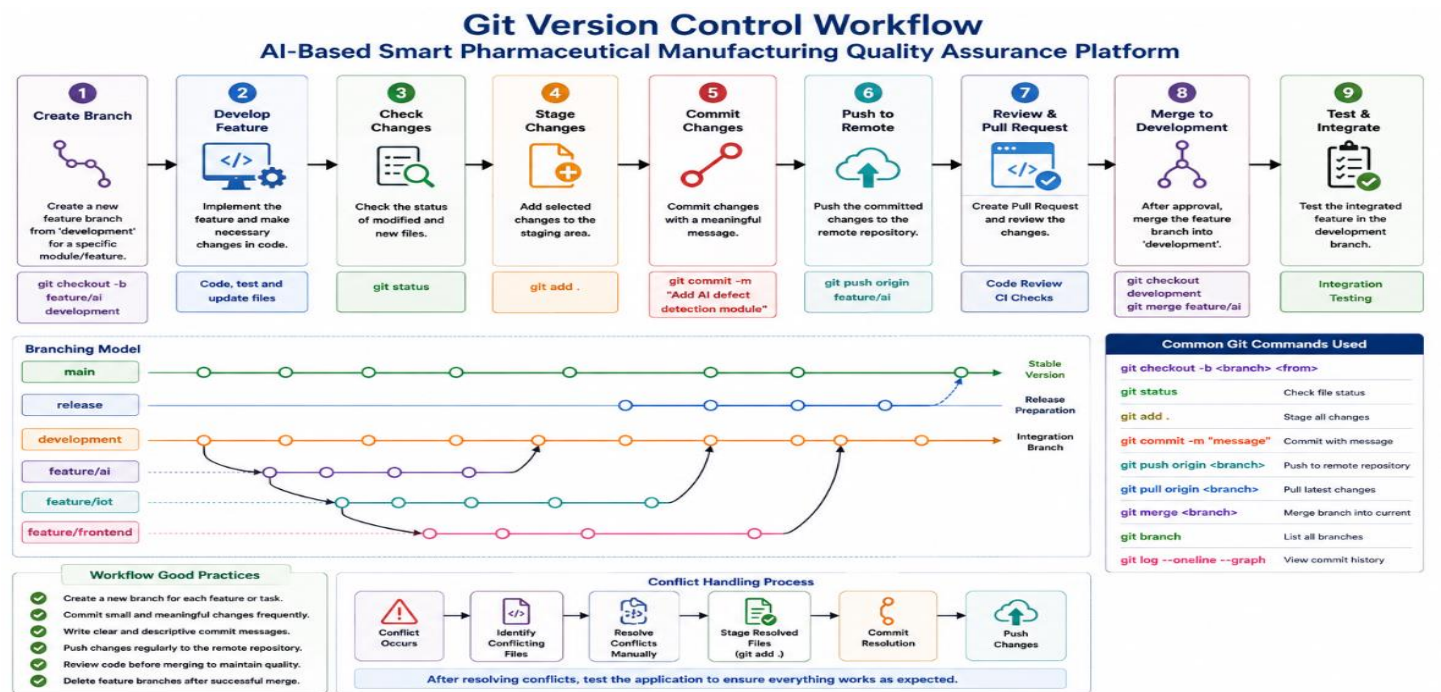


Figure 5: Git Version Control Workflow for the AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform

5.2 Creating and Committing Changes

After creating a feature branch, the developer makes the required changes to the project. Git provides commands to check, stage, and commit these changes.

Step 1: Check the Current Changes

```
git status
```

This command displays the modified, deleted, and untracked files in the working directory.

Step 2: Stage the Changes

```
git add .
```

This command adds the required changes to the Git staging area.

Step 3: Create a Meaningful Commit

```
git commit -m "Add AI defect detection module"
```

The commit records the staged changes in the Git repository with a meaningful description.

For other project modules, suitable commit messages can include:

```
Add IoT production monitoring module
Implement quality dashboard
Add batch traceability API
Add Docker configuration
Implement compliance reporting
```

Meaningful commit messages make the project history easier to understand and help developers identify what was changed at each stage.

Why Meaningful Commits Are Important

- Clearly identify the purpose of each change.
- Make debugging easier.
- Help track project development.
- Support code review.
- Make rollback easier when required.
- Improve collaboration between developers.

5.3 Branching and Merging

After completing a feature, the developer pushes the feature branch to the remote repository and prepares it for integration into the development branch. This process ensures that new functionality is reviewed and tested before becoming part of the integrated project.

Step 1: Switch to the Development Branch

```
git checkout development
```

This changes the current working branch to development.

Step 2: Merge the Feature Branch

For example, to merge the AI feature:

```
git merge feature/ai
```

This integrates the changes from feature/ai into the development branch.

Step 3: Check the Merge Result

```
git status
```

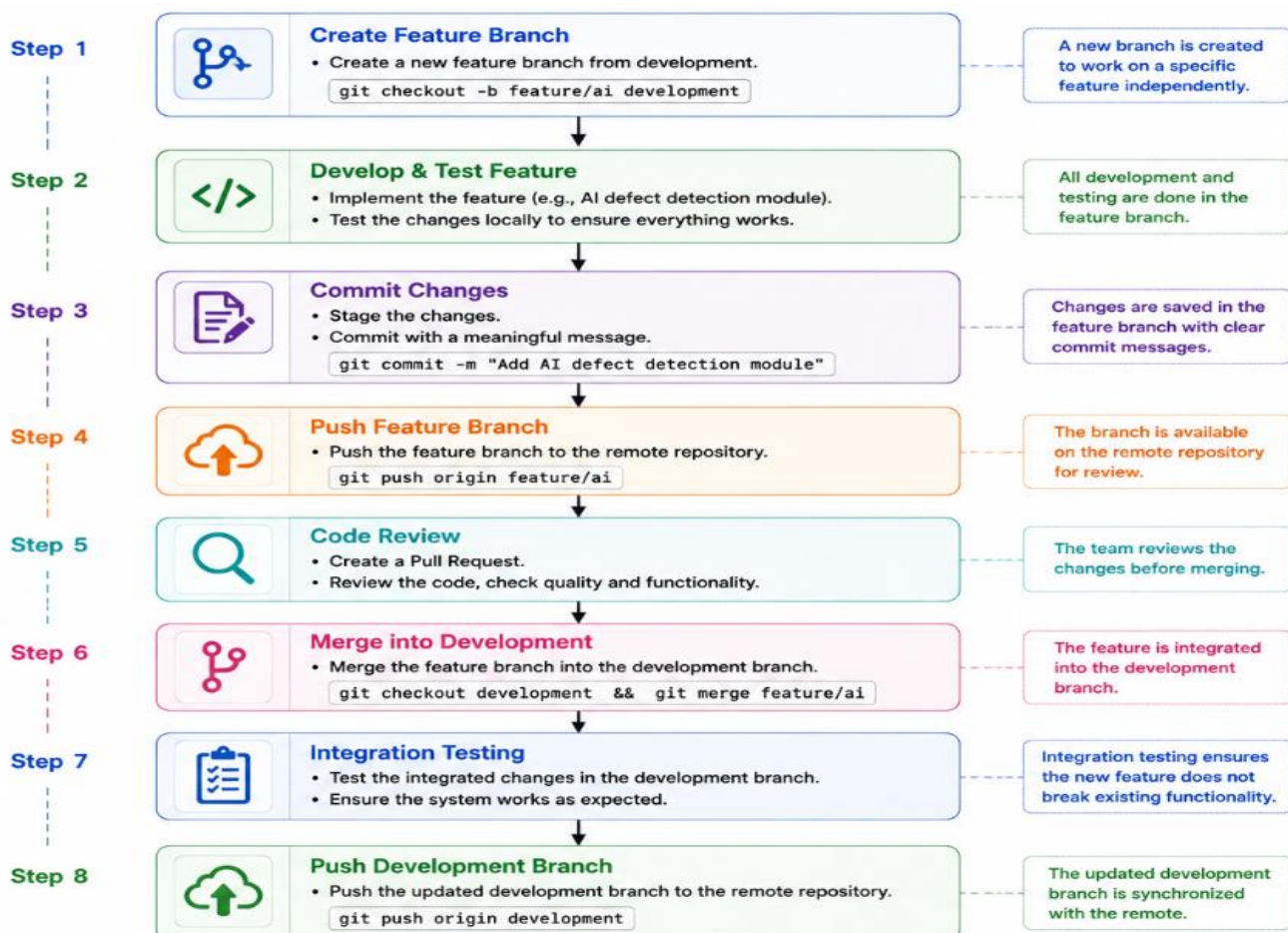
The command is used to verify whether the merge was completed successfully or whether any conflicts need to be resolved.

Step 4: Push the Updated Development Branch

```
git push origin development
```

This uploads the updated development branch to the remote repository.

Example Workflow



Importance of Branching and Merging

Branching and merging allow developers to work independently while keeping the main project stable. Features such as AI defect detection, IoT monitoring, frontend development, and Docker configuration can be developed separately and later integrated into the development branch.

5.4 Synchronization and Conflict Resolution

Synchronization keeps the local project repository updated with the latest changes available in the remote repository. In a collaborative development environment, developers should regularly pull changes before starting new work or merging their feature branches.

5.4.1 Pulling Latest Changes

The latest changes from the remote development branch can be retrieved using:

```
git pull origin development
```

This command downloads the latest changes and integrates them into the current local branch.

5.4.2 Pushing Changes

After completing and committing changes, the developer can upload the feature branch using:

```
git push origin feature/ai
```

After merging the feature into development, the updated branch can be synchronized using:

```
git push origin development
```

5.4.3 Handling Merge Conflicts

A merge conflict may occur when two developers modify the same section of a file in different branches. Git cannot automatically determine which version should be retained.

The conflict can be handled using the following process:

1. Identify the conflicting files.
2. Open the affected file.
3. Review the conflicting changes.
4. Select or combine the correct changes.
5. Remove the conflict markers.
6. Save the corrected file.
7. Stage the resolved file.
8. Commit the resolution.
9. Test the application.

Useful commands include:

```
git status
```

```
git add .
```

```
git commit -m "Resolve merge conflict"
```

5.4.4 Importance of Conflict Resolution

Proper conflict resolution prevents incorrect code from entering the integrated project. After resolving a conflict, the affected functionality should be tested to ensure that the changes from both developers work correctly together.

5.5 Version Control Workflow Summary

The Git workflow provides a systematic method for developing, reviewing, integrating, and maintaining the source code of the **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform**.

The complete workflow begins by creating a separate feature branch from the development branch. The developer then implements and tests the required functionality. After completing the changes, the modified files are checked and added to the staging area. A meaningful commit message is used to record the changes in the Git history.

The feature branch is pushed to the remote repository for review. After successful review and testing, the feature is merged into the development branch. Integration testing is then performed to ensure that the new feature works correctly with the existing modules. Finally, the updated development branch is synchronized with the remote repository.

The overall workflow can be summarized as:

Create Branch → Develop → Test → Stage → Commit → Push → Review → Merge → Integration Test → Synchronize

This workflow provides controlled development, clear version history, better collaboration, and reduced risk of introducing unstable changes into the project.

Key Benefits of the Workflow

- Maintains a clear history of project changes.
- Keeps feature development separate from stable code.
- Supports collaborative development.
- Enables code review before integration.
- Helps identify and resolve conflicts.
- Reduces the risk of unstable code reaching the main branch.
- Makes it easier to track and reverse changes when required.

6. Commit History Analysis

Git commit history provides a record of the changes made during the development of the **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform**. Each commit represents a specific change made to the project and helps developers understand how the system evolved over time.

6.1 Meaningful Commit History

The project should use clear and descriptive commit messages. Example commits for this project include:

Initial project setup
Create pharmaceutical quality dashboard
Add backend API structure
Implement AI defect detection module

- Add IoT production monitoring
- Implement batch traceability
- Add compliance reporting module
- Add Docker configuration
- Configure Docker Compose services
- Fix quality monitoring API
- Update project documentation

Each commit focuses on a specific logical change rather than combining many unrelated modifications.

6.2 Example Git Log

The project history can be viewed using:

```
git log --oneline
```

An example output is:

```
a82f91c Add Docker Compose configuration
b71e62a Implement compliance reporting module
c54d31e Add IoT production monitoring
d43e21f Implement AI defect detection
e21a98c Create pharmaceutical quality dashboard
f10a72b Initial project setup
```

The commit identifiers are unique references generated by Git for each commit.

6.3 Importance of Meaningful Commit Messages

Meaningful commit messages make the repository easier to understand and maintain. They help developers identify what functionality was added or modified without opening every changed file.

For example:

Good commit:

```
Implement AI defect detection
```

This clearly identifies the purpose of the change.

Poor commit:

```
changes
```

This does not provide enough information about what was changed.

Therefore, descriptive commit messages are important for project tracking, debugging, collaboration, code review, and future maintenance.

6.4 Commit History Benefits

A well-maintained commit history provides the following benefits:

- Tracks project development over time.
- Identifies when specific features were introduced.
- Helps locate the source of errors.
- Supports rollback to previous versions.
- Improves collaboration among developers.
- Makes code review easier.
- Provides a clear record of project progress.

7. Docker Environment Design

Docker is used to create a consistent environment for developing, testing, and deploying the **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform**. Since the proposed system contains multiple components such as the frontend, backend, AI model, IoT service, and database, containerization helps keep these components organized and isolated.

7.1 Need for Docker

The project may require different software libraries, frameworks, and dependencies for different components. Installing all these dependencies directly on a developer's computer can lead to compatibility problems.

Docker solves this problem by packaging an application together with its required dependencies into a container. This allows the application to run consistently across different environments.

For the proposed pharmaceutical quality assurance platform, Docker provides:

- **Environment consistency** between development and deployment.
- **Dependency isolation** for different services.
- **Portability** across systems that support Docker.
- **Simplified deployment** of multiple services.
- **Easy testing** in an isolated environment.
- **Scalability** of individual services.
- **Better maintainability** of the application.

7.2 Components to be Containerized

The following major components can be deployed as separate containers:

Container	Purpose
Frontend Container	Provides the pharmaceutical quality assurance dashboard and user interface.
Backend Container	Provides APIs, authentication, business logic, and application services.
AI Service Container	Performs AI-based defect detection and quality prediction.
IoT Service Container	Processes production data received from IoT sensors.
Database Container	Stores production, batch, inspection, and quality information.

7.3 Proposed Docker Environment

The proposed Docker environment follows a multi-container approach. Each major service operates inside its own container while communicating with other services through the Docker network.

The frontend communicates with the backend, while the backend communicates with the AI, IoT, and database services. This provides a clear separation between different application responsibilities.

7.4 Docker Networking

Docker networking allows the containers to communicate with one another within the application environment.

For example, the backend can communicate with the database using the database service name defined in Docker Compose rather than depending on a fixed IP address.

This makes the application easier to configure and deploy across different environments.

7.5 Docker Data Management

The database requires persistent storage so that important pharmaceutical production and quality information is not lost when a container is stopped or recreated.

Docker volumes can therefore be used to store database information outside the temporary container filesystem.

Example:

```
volumes:  
  pharma_data:
```

This volume can be attached to the database container for persistent data storage.

7.6 Docker Environment Benefits

The proposed Docker environment improves the project in several ways:

1. Each service can be developed independently.
2. Dependencies are isolated between services.
3. The complete application can be deployed consistently.
4. Individual services can be updated without modifying unrelated components.
5. Testing can be performed in a controlled environment.
6. Additional service instances can be created when required.
7. The application becomes easier to maintain and deploy.

8. Dockerfile Development

A **Dockerfile** is a text file containing a set of instructions used to build a Docker image for an application. For the proposed **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform**, the Dockerfile helps package the application and its required dependencies into a consistent container environment.

The Dockerfile can be used for the backend service, while similar Dockerfiles can be created for other services such as the frontend and AI service when required.

8.1 Backend Dockerfile

```
FROM python:3.11-slim
```

```

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "app.py"]

```

8.2 Explanation of Dockerfile Instructions

Instruction	Purpose
FROM python:3.11-slim	Selects Python 3.11 Slim as the base image for the application.
WORKDIR /app	Creates and sets /app as the working directory inside the container.
COPY requirements.txt .	Copies the Python dependency file into the container.
RUN pip install --no-cache-dir -r requirements.txt	Installs the required Python packages.
COPY . .	Copies the application source code into the container.
EXPOSE 8000	Indicates that the application uses port 8000.
CMD ["python", "app.py"]	Starts the application when the container runs.

8.3 Building the Docker Image

The Docker image can be created using:

```
docker build -t pharma-backend .
```

The option assigns the name `pharma-backend` to the image.

The available Docker images can be checked using:

```
docker images
```

8.4 Importance of the Dockerfile

The Dockerfile provides a repeatable method for creating the application environment. Instead of manually installing dependencies on every computer, the same Dockerfile can be used to build the application image consistently.

For this project, this helps ensure that the backend service has the required Python environment and dependencies whenever it is deployed.

9. Docker Compose Design

Docker Compose is used to define and manage multiple containers as a single application. Since the proposed **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform** consists of frontend, backend, AI, IoT, and database components, Docker Compose simplifies the process of configuring, starting, and managing these services together.

This section addresses Question 9 of the uploaded assignment, which requires the Docker Compose configuration and explanation of **services, networking, volumes, environment variables, and dependencies**.

9.1 Docker Compose Configuration

services:


```
frontend:
  build: ./frontend
  ports:
    - "3000:3000"
  depends_on:
    - backend
```

```
backend:
  build: ./backend
  ports:
    - "8000:8000"
  depends_on:
    - database
    - ai-service
```

```
ai-service:
  build: ./ai-model
```

```
iot-service:
  build: ./iot
```

```
database:
  image: postgres:15
  environment:
    POSTGRES_DB: pharma_qa
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: pharma_password
  volumes:
    - pharma_data:/var/lib/postgresql/data
```

```
volumes:
  pharma_data:
```

9.2 Services

Frontend Service

The frontend service provides the web-based dashboard through which authorized users can monitor production, view quality information, receive alerts, and access reports.

Backend Service

The backend service manages application logic, APIs, authentication, batch information, and communication between the frontend and other services.

AI Service

The AI service performs AI-based defect detection and quality prediction using the required Machine Learning and Computer Vision models.

IoT Service

The IoT service processes production information received from connected sensors and makes the collected data available to the backend and analytics modules.

Database Service

The database service stores production information, batch records, inspection results, quality information, and other application data.

9.3 Port Mapping

The frontend and backend services expose ports to allow users and other application components to access them.

ports:
- "3000:3000"

The frontend is accessible through port **3000**.

ports:
- "8000:8000"

The backend API is accessible through port **8000**.

9.4 Networking

Docker Compose creates a network that allows the different services to communicate with one another.

For example, the backend can communicate with the database using the database service name:

backend → database

Similarly, the backend can communicate with the AI service:

backend → ai-service

This service-based communication avoids depending on fixed container IP addresses.

9.5 Volumes

The database uses a Docker volume:

volumes:
pharma_data:

The volume is mounted inside the PostgreSQL container:

- pharma_data:/var/lib/postgresql/data

This provides persistent storage for pharmaceutical production and quality information. Database data can therefore remain available even when the database container is stopped or recreated.

9.6 Environment Variables

Environment variables are used to configure the database without placing configuration values directly inside application source code.

Example:

```
environment:
  POSTGRES_DB: pharma_qa
  POSTGRES_USER: admin
  POSTGRES_PASSWORD: pharma_password
```

For an actual deployment, sensitive credentials should preferably be stored in a `.env` file or a secure secrets-management system rather than committing them directly to the Git repository.

9.7 Service Dependencies

The `depends_on` instruction defines relationships between services.

For example:

```
backend:
  depends_on:
    - database
    - ai-service
```

This indicates that the backend service depends on the database and AI service.

The frontend also depends on the backend:

```
frontend:
  depends_on:
    - backend
```

This creates the following logical relationship:

```
Frontend
  ↓
Backend
 ↙  ↘
AI  Database
```

9.8 Running Docker Compose

The complete application can be built using:

```
docker compose build
```

The services can then be started using:

```
docker compose up
```

For background execution:

```
docker compose up -d
```

The running containers can be checked using:

```
docker ps
```

The services can be stopped using:

```
docker compose down
```

10. Container Deployment and Testing

The container deployment process is used to build, start, monitor, and test the different services of the **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform**. Docker Compose allows all required services to be deployed together and tested in a controlled environment.

10.1 Building the Docker Containers

After creating the Dockerfile and docker-compose.yml files, the Docker images can be built using:

```
docker compose build
```

This command reads the Docker Compose configuration and builds the required images for the frontend, backend, AI, and IoT services.

10.2 Starting the Containers

The complete application can be started using:

```
docker compose up
```

For running the services in the background:

```
docker compose up -d
```

The -d option allows the containers to continue running in detached mode.

10.3 Checking Running Containers

The following command is used to verify whether the required containers are running:

```
docker ps
```

The output should display the active containers along with their container IDs, names, ports, and status.

10.4 Checking Application Logs

Application logs can be viewed using:

```
docker compose logs
```

Logs help identify startup problems, connection errors, application failures, and communication issues between services.

Logs for a particular service can also be checked using:

```
docker compose logs backend
```

10.5 Application Testing

After starting the containers, the application should be tested to verify that the different services are communicating correctly.

The following tests can be performed:

1. Verify that all required containers are running.
2. Open the frontend application in a web browser.
3. Test user login and authentication.
4. Verify communication between the frontend and backend.
5. Test backend API functionality.
6. Verify AI service communication.
7. Verify IoT data processing.
8. Test database connectivity.
9. Verify that production and quality information is displayed correctly.
10. Check whether alerts are generated for abnormal conditions.
11. Verify that application logs do not contain critical errors.

10.6 Container Status Verification

The `docker ps` command can be used to confirm that the required services are active.

Example:

CONTAINER ID	IMAGE	STATUS
xxxxxxx	pharma-frontend	Up
xxxxxxx	pharma-backend	Up
xxxxxxx	pharma-ai	Up
xxxxxxx	pharma-iot	Up
xxxxxxx	postgres	Up

Note: The actual container IDs, names, and status should be taken from your own Docker execution.

10.7 Stopping the Containers

After completing the testing process, the containers can be stopped using:

```
docker compose down
```

This stops and removes the containers created by Docker Compose.

10.8 Deployment Testing Result

The deployment testing verifies whether the different components of the platform can operate together in a containerized environment. Successful execution should demonstrate that the frontend, backend, AI service, IoT service, and database can start correctly and communicate through the Docker network.

The testing process also helps identify configuration, networking, dependency, and application errors before deployment.

11. Benefits of Git and Docker

The combination of **Git and Docker** provides an effective development, version-control, and deployment approach for the **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform**. Git manages the source code and development history, while Docker provides a consistent environment for running the application and its services.

11.1 Benefits of Git

1. Version Control

Git records changes made to the project, allowing developers to track how the system has evolved over time.

2. Collaboration

Multiple developers can work on different modules such as AI, IoT, frontend, and backend without directly affecting each other's work.

3. Branching

Feature branches allow developers to develop new functionality separately before integrating it into the main project.

4. Change Tracking

Git provides a history of modifications, making it easier to identify who made a change and when it was made.

5. Rollback

If a new change causes an unexpected problem, the project can be restored to an earlier stable version.

6. Code Review

Branches and remote repositories support reviewing changes before they are merged into the stable project.

11.2 Benefits of Docker

1. Portability

The application can be deployed consistently on different systems that support Docker.

2. Environment Consistency

Docker ensures that the required software dependencies and configurations are packaged with the application.

3. Dependency Isolation

Each service can have its own dependencies without interfering with other services.

4. Simplified Deployment

Docker Compose allows multiple services to be started and managed together.

5. Scalability

Individual services can be scaled according to their workload.

6. Maintainability

Services can be updated or replaced independently without requiring changes to unrelated components.

11.3 Combined Benefits of Git and Docker

Using Git and Docker together provides a structured development and deployment workflow.

Git	Docker
Manages source code	Manages application environment
Tracks changes	Packages dependencies
Supports branching	Provides container isolation
Maintains project history	Provides deployment consistency
Supports collaboration	Supports portability
Enables rollback	Supports service scalability

Together, Git and Docker improve the overall development process by providing **version control, collaboration, reproducibility, portability, deployment efficiency, scalability, and maintainability.**

12. Challenges and Improvements

The development and deployment of the **AI-Based Smart Pharmaceutical Manufacturing Quality Assurance Platform** using Git and Docker may involve several technical and operational challenges. Identifying these challenges helps improve the reliability, maintainability, and scalability of the system.

12.1 Challenges

1. Managing Multiple Components

The platform contains frontend, backend, AI, IoT, database, and Docker components. Managing dependencies and communication between these components can increase project complexity.

2. AI Model Dependencies

AI and Machine Learning modules may require specialized libraries and larger computational resources. Managing these dependencies inside containers can increase image size and deployment requirements.

3. Large IoT Data Volume

Continuous monitoring of pharmaceutical manufacturing equipment can generate a large amount of sensor data. Efficient processing and storage are required to handle this data.

4. Git Merge Conflicts

When multiple developers modify the same files simultaneously, Git merge conflicts may occur. These conflicts need to be carefully resolved and tested.

5. Docker Networking

The frontend, backend, AI, IoT, and database containers must communicate correctly. Incorrect network or service configuration can cause communication failures.

6. Database Persistence

Important production and quality information must remain available even when containers are stopped or recreated. Therefore, proper Docker volume configuration is required.

7. Security of Configuration Data

Database passwords, API keys, and other sensitive configuration information should not be directly committed to the Git repository.

12.2 Improvements

The following improvements can make the development and deployment process more effective:

1. Use **protected Git branches** for stable project code.
2. Use **pull requests and code reviews** before merging changes.
3. Implement **automated testing** before merging feature branches.
4. Use **CI/CD pipelines** for automated build, test, and deployment.
5. Use **Docker image optimization** to reduce unnecessary image size.
6. Implement **container health checks** to identify unavailable services.
7. Use secure **environment variables or secrets management** for sensitive information.
8. Implement proper **database backup and recovery** mechanisms.
9. Use monitoring tools to track container and application performance.

10. Apply security scanning to source code and Docker images.

12.3 Future Enhancements

The platform can be further improved by integrating emerging technologies and advanced deployment practices.

Edge AI

AI processing can be performed closer to manufacturing equipment, allowing faster defect detection and reducing dependence on continuous cloud communication.

Kubernetes

Kubernetes can be introduced for large-scale container orchestration, service management, and automated scaling.

CI/CD Automation

A CI/CD pipeline can automatically build, test, and deploy the application whenever approved changes are pushed to the repository.

Blockchain-Based Traceability

Blockchain technology can be explored to provide tamper-resistant records for pharmaceutical batch traceability and supply-chain information.

Advanced Predictive Maintenance

Machine Learning can be used to predict equipment failures before they occur, helping reduce unexpected production interruptions.

Cloud-Based AI Deployment

AI models can be deployed through scalable cloud infrastructure to support centralized model management and future expansion.

12.4 Overall Improvement Strategy

The combination of improved Git practices, optimized Docker deployment, automated testing, CI/CD, security controls, and scalable cloud technologies can make the proposed platform more reliable and easier to maintain.

These improvements also provide a foundation for expanding the system to support additional manufacturing facilities, production lines, IoT devices, and AI-based quality assurance capabilities.

